

КОМАНДА ПРОЕКТА

TroneINC

Симулятор работы балансировщика нагрузки и платформа для сбора статистики по алгоритмам распределения запросов.

load-balancer-sim

WEB ARCHITECT

Ваня

DEVELOPER & QA

Влад

TEAM LEADER

Тимур

ANALYTIC

Погребницкий
Гоша

SOFTWARE ARCHITECT

Горбов Гоша

01

Проблемы и задачи

Зачем нужен симулятор балансировщика

Проблемы

Гонки в многопоточности

Многопоточность серверов и балансировщика ведёт к возможным гонкам, которые решаются только замедлением работы балансировщика.

Огромный набор параметров

Который нужно поддерживать, чтобы приблизить работу балансировщика к реальным условиям.

Задачи

1

Написать платформу, которая симулирует процесс работы балансера и собирает статистику.

2

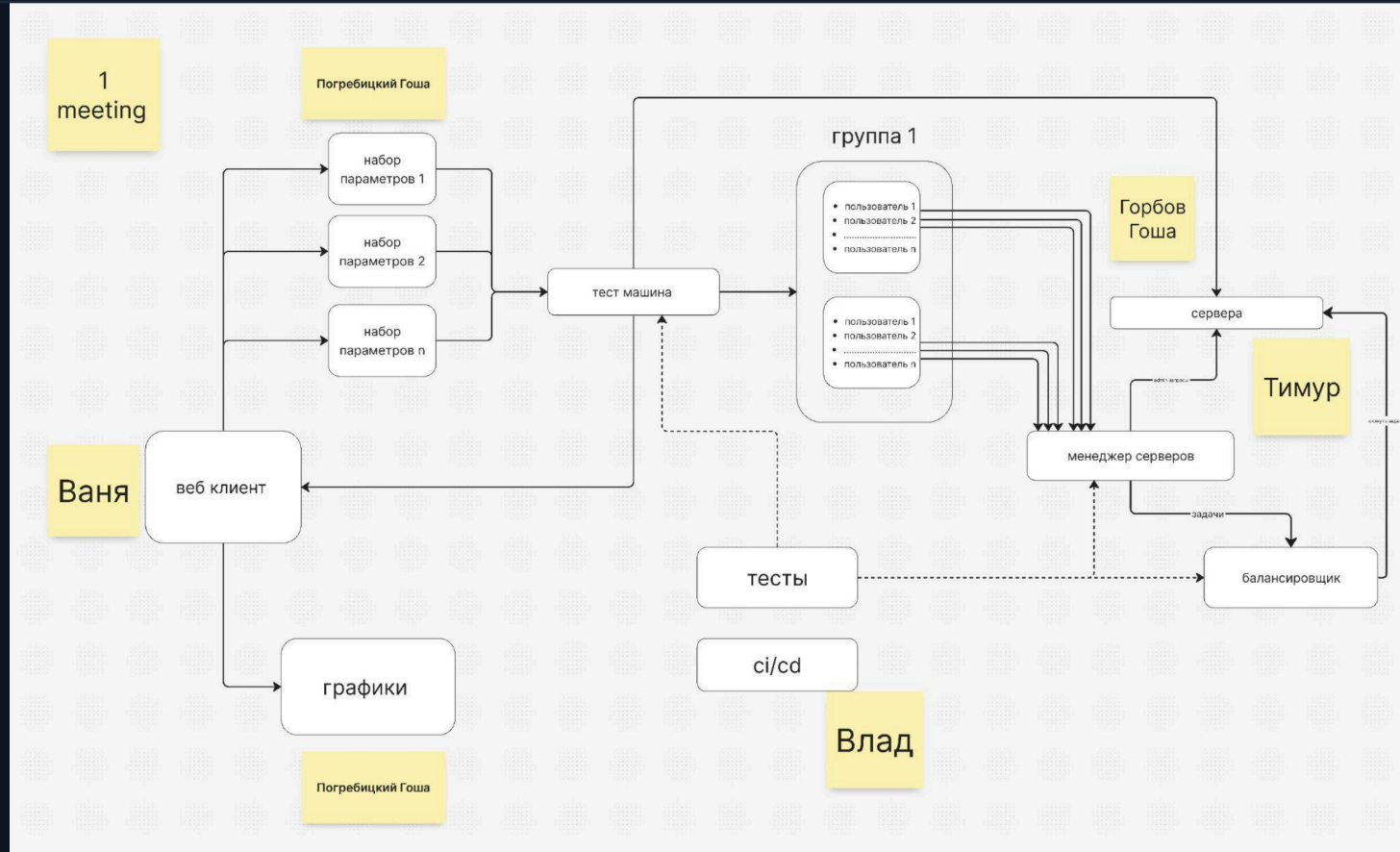
Написать веб-клиент, который удобно и гибко задаёт параметры симуляции и показывает статистику.

3

Собрать аналитику и сделать итоги.

Архитектура системы

Поток данных: веб-клиент → тест-машина → балансировщик → серверы



03

Четыре алгоритма балансировки

От простой ротации до распределения с учётом нагрузки и хеширования

01 Round Robin

Запросы идут по серверам по кругу: $1 \rightarrow 2 \rightarrow 3 \rightarrow \dots$ и снова. Не учитывает нагрузку, мощность и отклик серверов — зато минимальный оверхэд.

02 Weighted Round Robin

То же распределение по кругу, но с весами: сервер с весом 20 получает в 4 раза больше запросов, чем сервер с весом 5.

03 Least Connections

Новый запрос идёт на сервер с наименьшим числом активных соединений — учитывает реальную загрузку, но требует постоянного опроса.

04 Consistent Hashing

Серверы и ключи запросов хешируются на кольце; запрос идёт ближайшему по часовой стрелке серверу. Гарантирует session stickiness.

Конфигуратор симуляции с живым JSON

Имя

experiment

Профиль

custom

Seed

42

Длительность, мс

30000

Прогрев, мс

0

Лимит запросов

0

Воркеры менеджера

0

Описание

Серверы

+ сервер

weight	capacity	параллелизм	start_at_ms	crash_at_ms
1	1	1	0	∞
background_load: constant 0				
2	0.8	1	0	∞
background_load: constant 0				
3	0.6	1	0	∞
background_load: constant 0				

Группы клиентов

+ группа

count	sticky_scope	max_requests
3	none	0
inter_arrival_ms: exponential 100		
task_cost: constant 0.05		

Алгоритмы

По умолчанию запускаются все выбранные — по одному прогону на алгоритм.

☒ RoundRobin

☒ WeightRoundRobin

☒ LeastConnections

☒ ConsistentHashing

▶ Запустить

Сохранить пресет

Пример

Скачать JSON

Загрузить JSON

```
{  "name": "experiment",  "description": "",  "seed": 42,  "duration_ms": 30000,  "warmup_ms": 0,  "total_request_limit": 0,  "manager_workers": 0,  "servers": [    {      "weight": 1,      "capacity": 1,      "background_load": {        "type": "constant",        "value": 0      },      "max_parallel_requests": 1,      "start_at_ms": 0    },    {      "weight": 2,      "capacity": 0.8    },    {      "weight": 2,      "capacity": 0.6    }  ]}
```

Гибкая настройка симуляции: параметры эксперимента, серверы, группы клиентов и выбор алгоритмов — с живым JSON-конфигом.



Параметры: seed, длительность, прогрев, лимит запросов



Серверы: вес, capacity, параллелизм, фоновая нагрузка



Пресеты и экспорт / импорт JSON

Профили нагрузки

05

Шесть сценариев моделируют разные состояния системы. У большинства — варианты **easy / medium / hard** по стоимости запросов.

01 Uniform

Периоды низкой активности — равномерные запросы малой вычислительной стоимости.

02 Variable Request Cost

Равномерная частота запросов, но варьирующаяся вычислительная стоимость.

03 Server Heterogeneity

Серверы разной мощности — задаётся параметром для каждого профиля.

04 Burst

Резкий всплеск пользовательской активности.

05 Churn

Состав серверов часто меняется прямо во время теста.

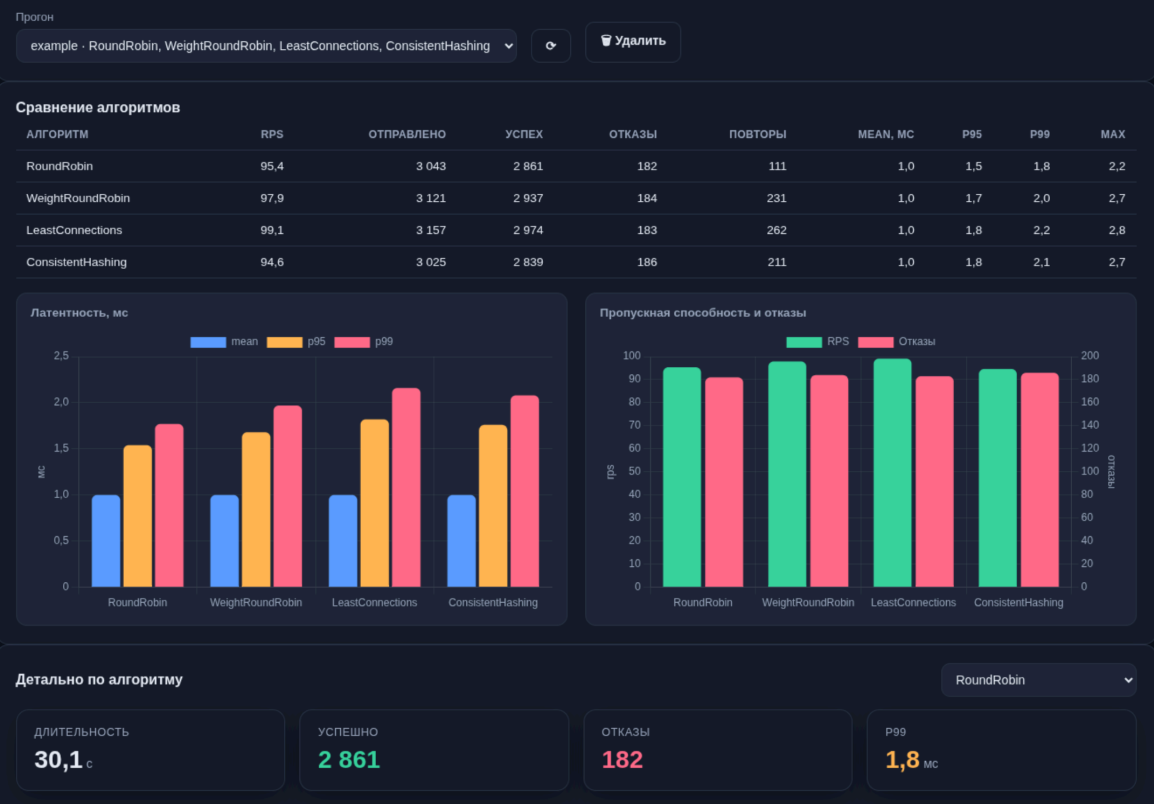
06 Session Stickiness

Повторные запросы одного пользователя должны идти на тот же сервер.

Результаты: сравнение

сравнение алгоритмов

Прогон четырёх алгоритмов на одном сценарии — по латентности и пропускной способности



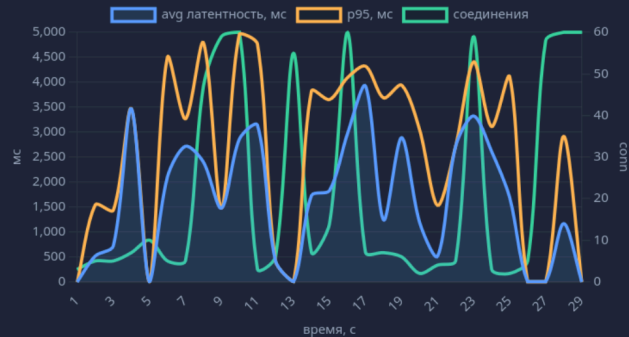
07

Детально по прогону

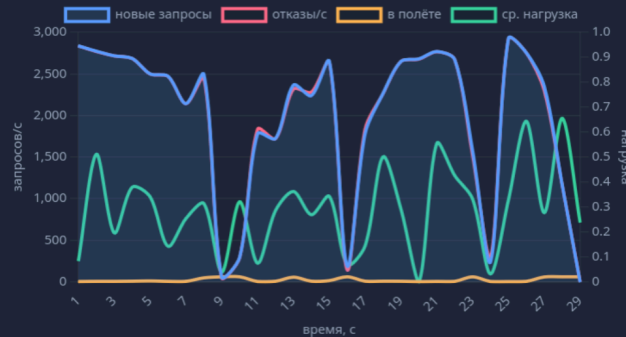
Таймлайн латентности и соединений, гистограмма, нагрузка серверов avg/peak

Отказы (58 793): crashed 4 395 · overloaded 54 398 · timeout 0

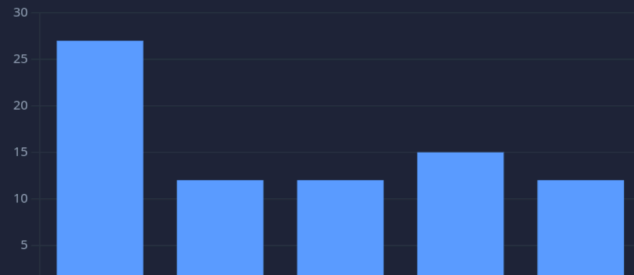
Таймлайн: латентность и соединения



Таймлайн: запросы и нагрузка



Гистограмма латентности



Нагрузка серверов (avg/peak)



avg латентность, p95 и соединения во времени

разбивка по серверам и группам клиентов

08

Производительность по сложности

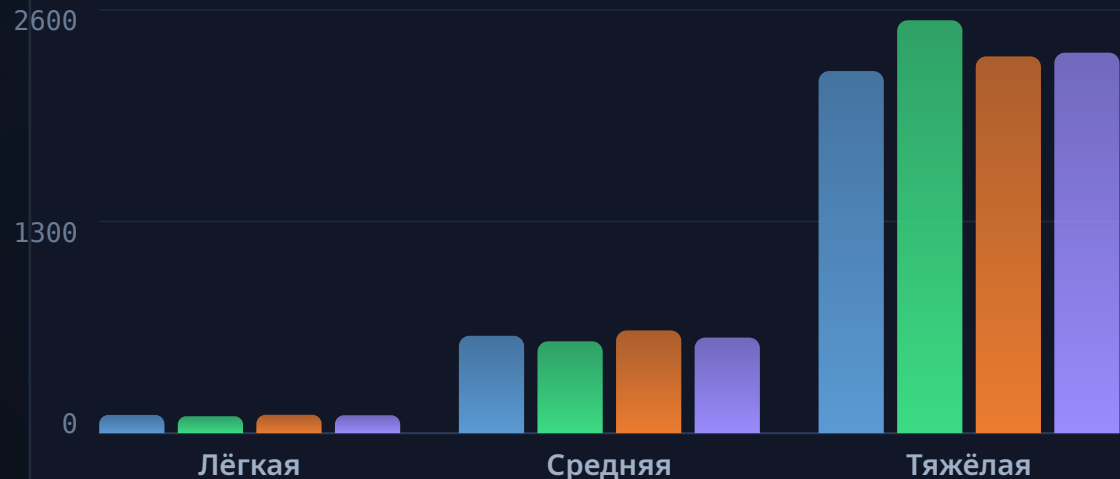
Усреднение по 26 сценариям. На тяжёлой нагрузке пропускная способность падает почти в **10×**, а p99 растёт до **2.5 с**.

■ Round Robin ■ Weighted RR ■ Least Conn. ■ Consistent H.

Пропускная способность запросов/с



Латентность p99 мс



09

Баланс нагрузки серверов

Disparity — неравномерность распределения по серверам. **Ниже — равномернее.**

Round Robin



1.05

Weighted
Round Robin



0.65

РОВНЕЕ ВСЕХ

Least
Connections



0.99

Consistent
Hashing



0.98

Weighted Round Robin

распределяет запросы ровнее всех — disparity 0.65 против ~1.0 у остальных алгоритмов.

Тяжёлая нагрузка

пропускная способность проседает с ~66 до ~6.5 запросов/с — почти в 10 раз.

p99 латентность

на hard-сценариях растёт до 2.2–2.5 с против ~110 мс на лёгкой нагрузке.

Гипотезы → реальность

Что ожидали от каждого алгоритма и что показали прогоны

АЛГОРИТМ	ГИПОТЕЗА	ЧТО ПОЛУЧИЛИ
■ Round Robin	Хорош на простых сценариях, проигрывает на сложных из-за игнорирования нагрузки.	ПОДТВЕРДИЛОСЬ Подтвердилось полностью: эффективен на простых, неэффективен на сложных.
■ Weighted Round Robin	Хорош на простых и средних; уязвим на Variable Request Cost — может перегрузить сильные серверы.	СИЛЬНЕЕ ОЖИДАНИЙ Оказался эффективен и на сложных задачах — заметно лучше ожиданий.
■ Least Connections	Лучший на самых сложных сценариях, нагружает серверы пропорционально мощности.	С ОГОВОРКОЙ Действительно силен на сложных — но склонен перегружать самый слабый сервер.
■ Consistent Hashing	Решает проблему stickiness и не сильно медленнее при churn.	В ОСНОВНОМ ВЕРНО

11 Выводы

■ Round Robin

Прост и эффективен на простых задачах, но проваливается на сложных.

■ Weighted Round Robin

Неожиданно силен: хорош и на простых, и на сложных, с самым ровным балансом нагрузки.

■ Least Connections

Отлично держит сложные сценарии, но имеет опасное свойство — перегружает самый слабый сервер.

■ Consistent Hashing

Гарантирует session stickiness; качество зависит от удачности хеширования.